SEAC□□□

“ ”

“ ” “ ”

SEAC
Structure- and Evidence-Aware Chunking

1 –

2 –

3 10 89
66981 125

4 0.05

2 3
4 SEAC 5 6
7 8 9
10

2 □□□□

2.1 □□□□□□□□□□□

[2] Dense X Retrieval
[5] Late
Chunking
[6] RAPTOR [7]
SEAC

SEAC

2.2 □□□□□□□

Docling [8] “
” “ ”

2.3 □□□□□□□□□□□

[9-10]
– –
“ ”
LangChain
LlamaIndex [11-12]

2.4 □□□□□□□□□

SEAC “ ” OCR

MRR
MRR

“ ”

3 □□□□

3.1 □□□□□□□□□

$D = (u_1, u_2, ..., u_n)$.

u_i

h r_j

$t(r_j) = header(h) \Vert row(r_j)$.

3.2 □□□□□□□□□□

q G(q) z(q) e_i G(q) 1 G(q) “ ” z(q) “ ” k I_h I_path I_group

$$d_i = 1 - \cos(e_i, e_{i+1}).$$

$$s_i = \min\{1, I_h + 0.7I_{path} + 0.5Igroup\}.$$

$$k_i = \min\{1, 10|\rho(u_i) - \rho(u_{i+1})|\}.$$

$$p_i = 0.35I_{same-path} + 1.0I_{same-group}.$$

3.3 □□□□

RQ1 SEAC “ ” 7 RQ2 SEAC RQ3 RQ4 RQ5

$$b_i = 0.45d_i + 0.30s_i + 0.25k_i - p_i.$$

0.8

$$\theta_D = Q_{0.8}(b) + 0.08r_t - 0.05r_h.$$

r_t r_h

4 SEAC □□

4.1 □□□□□□□□□□□□

SEAC `document_id` `block_id` `unit_id` “ ”

4.3 □□□□□□

512 BGE 320 0.03 224 48 b_i

4.2 □□□□□□

(u_i,u_{i+1}) d_i s_i k_i p_i BGE-M3

4.4 □□□□□□□□□□

112 176

k_i 10 1

“ ” SEAC V tokens(u)

p q S_p S_c $\Sigma_{v \in V}$ count(v,u) / max(1, tokens(u)).

“ ”“ ”“ / ” “ / ” $S(p,q) = 0.35S_p(q) + 0.65 \max S_c(q)$.

UTF-8 1

□□□□ □□·□□·□□	→	□□□□□ □/□□/□□□ □	→	□□□□□ □□□□·□□ □□	→	□□□□ □□□□□□	→	□□□□ □□□+□□□ □

1 SEAC

4.5 □□□□□□□□□□

n SEAC n n-1
O(n)

SHA-256 Python

4.6 □□□□□□□□□

“ →
→TASK_ITEM” “ ”
“STATUS VARCHAR 16 ”
“STATUS VARCHAR 16 ” “
” “ ” STATUS
SEAC “ → →TASK_ITEM
STATUS VARCHAR
16 ” `unit_id`

`trigger_unit_id` `parent_chunk_id`
`source_block_id` “ ”

G(q) G(q)

STATUS
SEAC

“TASK_ITEM” –
512

5 □□□□□□

5.1 □□□□□□□□□□□□

10 P01—P10 89

DOCX XLSX 6
PDF PPTX

P01—P02 P03
P04—P10

1

1

□□	□□□	□□□	□□□□	□□□	□□□□□
	2	15	9639	21371	418541
	1	7	3075	5193	103562
	7	67	19127	40417	1155797
	10	89	31841	66981	1677900

125 125

5.2 □□□□□□

P01 4783 P02—P10
300 SEAC
100
Cohen κ
0.70 [13]

κ
P08 κ=0.650
5566 77.5% 387
5.4% 1230 17.1% F1

20260804

κ

/ κ P02 0.61/0.87 P03 0.56/0.76 P04
0.81/0.89 P05 0.66/0.96 P06 0.42/0.91 P07
0.75/0.98 P08 0.68/0.65 P09 0.75/0.72 P10
0.77/0.76 κ κ
P08 “ ”

`HUMAN_VERIFIED`
`RULE_4_1`
`LLM_PRELABEL`
NOISE F1

5.3 □□□□□□□

215 195
10
P10 11 15
P04—P09 125

P04—P07 P09 P10 1368 2

9 195

`NO_RISK` `ANONYMIZED` “ ” `CLEARED`

262 71 124 67

12

1—8 18

12

2

□□	□□□	□□□	□□□□	□□□□□
P04	17	8532		22
P05	17	8166		17
P06	12	10122		22
P07	8	2496		23
P08	5	4369	κ=0.650	22
P09	4	5324		19
P10	4	1408		11<15

5.4 □□□□□□□□□□

BGE-M3 512 1 2 3 4 5 LangChain RecursiveCharacterTextSplitter 1.1.2 6 LlamaIndex SentenceSplitter 0.14.23 7 SEAC 8 SEAC [16]

LumberChunker [17-19] BERGEN [20] SEAC

SEAC

5.5 □□□□□□□□□□

NOISE F1 MRR Hit@1/3/5 MRR Hit@1/3/5 MRR Hit@5

10000 [14] SEAC LlamaIndex MRR 6 64 Holm [15]

“ ” Hit@k [21]

Python 3.12.3 `sentence-transformers` 5.6.0 `langchain-text-splitters` 1.1.2 `llama-index-core` 0.14.23 Linux ARM64 NVIDIA CUDA 16 20260804 3 SEAC P01—P10

3 SEAC

	□□	□□
	BGE-M3	
	512/320/224/48	
	112/176	
	0.45 0.30 0.25	
	0.8 +0.08 −0.05	
	0.35/0.65	
	10000	
	Python 3.12.3 Linux ARM64 CUDA	

5.6 □□□□□□□□□□

P04 —P10 QA

`industrial-seac-formal-analysis-v1.1`

5.7 □□□□□□□□□□□□□□□□□□□□

“ ” “ ”

“ ”

“ ”

6 □□□□

6.1 □□□□□□□□□□

4 SEAC F1 0.5673 95% [0.4708, 0.6488] SEAC 0.5032 SEAC 6 Holm p=0.4375 F1 0 125 SEAC MRR 0.7937 95% [0.7060, 0.8772] MRR 0.8561 Hit@5 0.9520 SEAC MRR 0.7290 0.0647 LlamaIndex MRR +0.1003 95% [0.0042, 0.2008] p=0.0625 0 0.05 6 64

4

□□	□□ F1	□□ MRR	Hit@1	Hit@5	□□ MRR
	0.0000	0.5158	0.3680	0.7200	0.1350
	0.0369	0.6086	0.4480	0.8080	0.7072
	0.5032	0.7290	0.6080	0.8880	0.8211
	0.3843	0.6999	0.5680	0.8640	0.8121
LangChain	0.0054	0.6460	0.4960	0.8480	0.7329
LlamaIndex	0.0920	0.6934	0.5520	0.8880	0.7586
SEAC	0.5673	0.6796	0.5280	0.8800	0.7770
SEAC	0.5673	0.7937	0.6800	0.9520	0.8561

6.2 □□□□□□

SEAC

“SEAC

”

5 SEAC MRR SEAC P04 P05 P07 P08 P09 P06 P05 P06 0.0402 P06

5 SEAC LlamaIndex

□□	□□□	SEAC MRR	LlamaIndex MRR	□□
P04	22	0.6914	0.5033	+0.1881
P05	17	0.8627	0.6275	+0.2353
P06	22	0.6805	0.7206	−0.0402
P07	23	0.8080	0.7360	+0.0719
P08	22	0.8386	0.7224	+0.1162
P09	19	0.9123	0.8557	+0.0566

6.3 □□□□□□

5

MRR Hit@5 1.0000 MRR 0.9839 MRR MRR 0.5429 Hit@5 0.8000 0.7323 0.2516

6

6 SEAC

□□□□	□□	□□ MRR	Hit@5	□□ MRR
	17	0.8211	0.9412	0.8211
	10	0.7833	1.0000	0.7833
	16	0.9167	0.9375	0.9167
	31	0.7323	1.0000	0.9839
	23	0.7867	0.9130	0.7867
	9	0.8370	1.0000	0.8370
	9	1.0000	1.0000	1.0000
	10	0.5429	0.8000	0.5429

6.4 □□□□

0.8640 MRR 0.8289 7 SEAC 0.8129 Hit@5 0.9280 MRR MRR 0.7937 0.6853 Hit@5 0.9520 0.0042

7

□□	□□ F1	□□ MRR	Hit@5	□□□□ MRR □□
SEAC	0.5673	0.7937	0.9520	–
	0.5698	0.7937	0.9520	+0.0000
	0.5639	0.7959	0.9520	+0.0021
	0.5718	0.7979	0.9520	+0.0042
	0.5731	0.7959	0.9520	+0.0022
	0.5673	0.8129	0.9280	+0.0192
	0.5673	0.6853	0.8640	−0.1084
	0.5673	0.8289	0.9520	+0.0352

SEAC “ ” Holm p=1.0000 0.1084

6.5 □□□□

6 8891 SEAC 48912 5.50 SEAC 15.18 ms 20.93 ms 5.74 ms 8 SEAC

8

□□	□□□	□□□□/s	□□□□/s	□□□□/95 □□/ms
	8891	0.25	139.46	15.18/18.40
SEAC	48912	112.11	351.12	20.93/27.28

0.9520 4.8%

MRR MRR

6.6 □□□□□□□□□□

“ ” “ MRR 0.6934 0.7937 ” 5 1 20 10 Hit@5 SEAC Hit@5

F1 MRR
SEAC -only 0.5673 F1
MRR F1

P06
P06 SEAC
5.50

“ ”

SEAC 0.8289

7 □□□□□□□□□□□□□□□□□

7.1 □□□□□□□□□□

SEAC
“ ”
“ ” “
”

`document_id/block_id/unit_id`

SEAC “ — —
”

—

7.2 □□□□□□□□□

MRR MRR

“ ” —

SEAC

7.3 □□□□□□□□□□□□

SEAC

7.4 □□□□□□

SEAC 5.50

MRR MRR

“ ”

7.5 □□□□□□□□□□

“ — — — — ”

SEAC

SEAC

8 □□

8.1 □□□□□□□□□

“ ” SEAC
MRR 0.6796 0.7937 0.6853

MRR

8.2 □□□□□□□□□□□□□□□□

0.8289 MRR

MRR Hit@5
“ ”

SEAC SEAC
0.8289

8.3 □□□□□□□

6

9 □□□□□□□□□□□□

9.1 □□□□□□□□

F1
77.5%
SEAC P08

66981
F1

9.2 □□□□□□□□

6
64

p=0.0625

PDF 6
PDF/PPTX
—

9.3 □□□□□□□□□□

10 □□

SEAC

10 SEAC 125
MRR 0.7937 Hit@5 0.9520 SEAC
LlamaIndex p=0.0625

“ ” “ ” MRR
0.6853 0.8289
SEAC 5.50

SEAC

□□ A □□□□□□□□□

— —

1
“
+ ”
`unit_id` `document_id` `block_id`

2 512

3

d_i s_i k_i p_i b_i

4 0.8

0.03

5

– JSON 0.7937

2 0.9520 0.7290 0.1003 [0.0042,0.2008]

p=0.0625 0.6853 0.8289 5.50

6

112 176 PDF Word

7

10 0

G(q)

z(q)

8

MRR

□□B □□□□□□□□□□□

/

□□C □□□□□□□□□

SHA-256 Python A1 10 BGE

WordPiece

P08 κ=0.650 P10 11<15 P04—P10

A1

□□	□□	□□	□□□	□□	□□□□	□□	□□□
P01		7	4790	10574	194119	839	6623
P02		8	4849	10797	224422	986	6934
P03		7	3075	5193	103562	412	2530
P04		17	4180	8532	301754	474	4826
P05		17	3886	8166	228920	464	4744
P06		12	5623	10122	255278	758	5257
P07		8	1564	2496	59925	172	1104
P08		5	1206	4369	123307	198	3361
P09		4	1606	5324	133710	329	4047
P10		4	1062	1408	52903	62	408
	–	89	31841	66981	1677900	4694	39834

A2 κ≥0.70

P08

A2

□□	□□□ κ	□□□ κ	□□□□□/%	□□□□
P02	0.61	0.87	94.0	
P03	0.56	0.76	88.8	
P04	0.81	0.89	96.3	
P05	0.66	0.96	98.8	
P06	0.42	0.91	97.2	
P07	0.75	0.98	98.8	
P08	0.68	0.65	84.5	
P09	0.75	0.72	87.6	
P10	0.77	0.76	89.9	

A3 4 SEAC -
only

A3

□□	□□□	□□□	F1	F1 □ 95%□□□□
	0.0000	0.0000	0.0000	[0.0000, 0.0000]
	0.0402	0.0340	0.0369	[0.0051, 0.0783]
	0.3611	0.8298	0.5032	[0.4108, 0.5859]
	0.2920	0.5617	0.3843	[0.2779, 0.4800]
LangChain	0.0074	0.0043	0.0054	[0.0000, 0.0265]
LlamaIndex	0.4615	0.0511	0.0920	[0.0367, 0.1570]
SEAC	0.4321	0.8255	0.5673	[0.4704, 0.6486]
SEAC	0.4321	0.8255	0.5673	[0.4708, 0.6488]

A4 Hit@3 Hit@5 SEAC
Hit@3 0.9040 5 Hit@5
Hit@5 0.9520

A4

□□	□□ MRR	Hit@1	Hit@3	Hit@5	□□ MRR	□□ Hit@5
	0.5158	0.3680	0.6240	0.7200	0.1350	0.1840
	0.6086	0.4480	0.7200	0.8080	0.7072	0.8480
	0.7290	0.6080	0.8320	0.8880	0.8211	0.9040
	0.6999	0.5680	0.8240	0.8640	0.8121	0.8960
LangChain	0.6460	0.4960	0.7440	0.8480	0.7329	0.8720
LlamaIndex	0.6934	0.5520	0.8400	0.8880	0.7586	0.8960
SEAC	0.6796	0.5280	0.8160	0.8800	0.7770	0.8960
SEAC	0.7937	0.6800	0.9040	0.9520	0.8561	0.9520

□□□□□□□□□□□

OpenAI Codex

□□□□

[1] Lewis P, Perez E, Piktus A, et al. Retrieval-augmented generation for knowledge-intensive natural language processing tasks. Advances in Neural Information Processing Systems, 2020, 33: 9459-9474.

[2] Karpukhin V, Oğuz B, Min S, et al. Dense passage retrieval for open-domain question answering. In: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing. Online: Association for Computational Linguistics, 2020: 6769-6781.

[3] Reimers N, Gurevych I. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th

International Joint Conference on Natural Language Processing. Hong Kong: Association for Computational Linguistics, 2019: 3982-3992.

[4] Chen J, Xiao S, Zhang P, et al. M3-Embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. In: Findings of the Association for Computational Linguistics: ACL 2024. Bangkok: Association for Computational Linguistics, 2024: 2318-2335.

[5] Chen T, Wang H, Chen S, et al. Dense X retrieval: What retrieval granularity should we use? In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing. Miami: Association for Computational Linguistics, 2024: 15159-15177.

[6] Günther M, Mohr I, Williams D J, et al. Late chunking: Contextual chunk embeddings using long-context embedding models. arXiv preprint arXiv:2409.04701, 2024.

[7] Sarthi P, Abdullah S, Tuli A, et al. RAPTOR: Recursive abstractive processing for tree-organized retrieval. In: Proceedings of the Twelfth International Conference on Learning Representations. Vienna: OpenReview, 2024.

[8] Auer C, Dolfi M, Carvalho A, et al. Docling technical report. arXiv preprint arXiv:2408.09869, 2024.

[9] Guo J L C, Steghöfer J P, Vogelsang A, Cleland-Huang J. Natural language processing for requirements traceability. arXiv preprint arXiv:2405.10845, 2024.

[10] Ali S J, Naganathan V, Bork D. Establishing traceability between natural language requirements and software artifacts by combining retrieval-augmented generation and large language models. In: Conceptual Modeling. Cham: Springer, 2024: 295-314.

[11] LangChain. langchain-text-splitters 1.1.2. Python Package Index, 2026.

[12] LlamaIndex. SentenceSplitter, llama-index-core 0.14.23. Python Package Index, 2026.

[13] Cohen J. A coefficient of agreement for nominal scales. Educational and Psychological Measurement, 1960, 20(1): 37-46.

[14] Efron B. Bootstrap methods: Another look at the jackknife. The Annals of Statistics, 1979, 7(1): 1-26.

[15] Holm S. A simple sequentially rejective multiple test procedure. Scandinavian Journal of Statistics, 1979, 6(2): 65-70.

[16] Wang X, Wang Z, Gao X, et al. Searching for best practices in retrieval-augmented generation. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing. Miami: Association for Computational Linguistics, 2024: 17716-17736.

[17] Duarte A V, Marques J, Graça M, et al. LumberChunker: Long-form narrative document segmentation. In: Findings of the Association for Computational Linguistics: EMNLP 2024. Miami: Association for Computational Linguistics, 2024: 6473-6486.

[18] Wang Z, Li Y, Zhao Y, et al. Document segmentation matters for retrieval-augmented generation. In: Findings of the Association for Computational Linguistics: ACL 2025. Vienna: Association for Computational Linguistics, 2025: 8063-8075.

[19] Zhao J, Zhang Y, Yang X, et al. MoC: Mixtures of text chunking learners for retrieval-augmented generation system. In: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics. Vienna: Association for Computational Linguistics, 2025: 5172-5189.

[20] Rau D, Wang S, Déjean H, et al. BERGEN: A benchmarking library for retrieval-augmented generation. In: Findings of the Association for Computational Linguistics: EMNLP 2024. Miami: Association for Computational Linguistics, 2024: 7640-7663.

[21] Liu N F, Lin K, Hewitt J, et al. Lost in the middle: How language models use long contexts. Transactions of the Association for Computational Linguistics, 2024, 12: 157-173.

[22] Mucha J, Kaufmann A, Riehle D. A systematic literature review of pre-requirements specification traceability. Requirements Engineering, 2024, 29: 119-141.

[23] Wang B, Zou Z, Wan H, et al. An empirical study on the state-of-the-art methods for requirement-to-code traceability link recovery. Journal of King Saud University—Computer and Information Sciences, 2024, 36: 102118.

Background

Large language models are increasingly embedded in software engineering workflows, but industrial software differs from open-domain text generation in the density of constraints, the heterogeneity of artifacts, and the consequences of unsupported outputs. Requirements, interface specifications, database dictionaries, detailed designs, technical agreements, and meeting records jointly define system behavior. The same term may refer to different modules or versions, while critical evidence is often expressed in a table row whose meaning depends on its header and section path. A retrieval-augmented generation system therefore needs more than semantically similar passages: it needs evidence that remains attributable to the correct artifact location.

Document chunking determines the units visible to a retriever. Fixed windows can cut through a requirement or table. Generic recursive splitters follow separators but do not understand engineering coupling. Heading-only splitters preserve sections yet may return excessive context for field-level questions. Semantic splitters may identify topical shifts while separating a data row from its schema. These errors propagate to downstream generation, citation, consistency checking, and change-impact analysis. Long-context models do not remove the problem because irrelevant context increases cost and evidence can still be overlooked or confused.

SEAC addresses this representation layer. It normalizes paragraphs, headings, and table rows into stable source units, constructs coherent parent chunks with deterministic boundary rules, and indexes fine-grained children that retain heading paths and table headers. A child is used for matching, while its parent provides interpretation context. The retrieval record preserves the exact document, block, and unit identifiers that produced the result. This separation enables two complementary evaluations: whether returned text contains the supporting span and whether it originates from an accountable source unit.

The industrial study follows a frozen protocol because project templates and terminology create clustered observations. Development, validation, and

locked-test projects are separated. Questions and boundary references are reviewed before locked metrics are released. Statistical inference resamples or exchanges labels at the project level rather than treating all questions as independent. The study reports negative results and resource costs alongside aggregate gains. In particular, the available evidence supports header-aware table-row retrieval, but it does not validate every component of the full scoring design or establish general superiority with only six independent retrieval projects.

For intelligent industrial software engineering, provenance-aware retrieval can support requirements extraction, interface and database consistency checking, test generation, defect review, and audit logs. It must nevertheless be combined with document-level access control, versioned parent-child mappings, privacy review, and human responsibility. Retrieved evidence can improve the verifiability of a large-model workflow, but it does not by itself authorize automated decisions or guarantee that a generator will faithfully use the evidence.